C++ Namespaces

A namespace in C++ is a way to organize code and prevent name conflicts. It groups related classes, functions, variables, and other identifiers under a unique name.

Why use namespaces?

Imagine two libraries both have a function named `print()`.

Without namespaces:

```
void print() {
    cout << "Library 1";
}

void print() {
    cout << "Library 2";
}
```

This causes a redefinition error because the compiler cannot distinguish between the two.

With namespaces:

```
#include <iostream>
using namespace std;

namespace Library1 {
    void print() {
        cout << "Library 1\n";
    }
}

namespace Library2 {
    void print() {
        cout << "Library 2\n";
    }
}

int main() {
    Library1::print();
    Library2::print();

    return 0;
}
```

Output:

Library 1
Library 2

Here, `::` is the scope resolution operator, used to access members of a namespace.

Syntax

```
namespace NamespaceName {
    int x = 10;

    void display() {
        cout << x;
    }
}
```

Access members like this:

```
NamespaceName::display();
```

Using `using namespace`

Instead of writing the namespace name repeatedly:

```
using namespace NamespaceName;

display();
```

Example:

```
#include <iostream>
using namespace std;

namespace Test {
    int x = 100;
}

using namespace Test;

int main() {
    cout << x;
}
```

Output:

100

Using a specific member

Instead of importing the whole namespace:

```
using NamespaceName::memberName;
```

Example:

```
#include <iostream>
using namespace std;

namespace Test {
    int x = 50;
    int y = 100;
}

using Test::x;

int main() {
    cout << x;
}
```

Only `x` is imported.

Nested namespaces

Namespaces can be inside other namespaces.

```
#include <iostream>
using namespace std;

namespace A {
    namespace B {
        void show() {
            cout << "Nested Namespace";
        }
    }
}

int main() {
    A::B::show();
}
```

Output:

Nested Namespace

C++17 shortcut

```
namespace A::B {
    void show() {
        cout << "Nested Namespace";
```

```
    }
}
```

Anonymous namespace

An anonymous namespace has no name.

```
namespace {
    int value = 10;
}

int main() {
    cout << value;
}
```

Members in an anonymous namespace are accessible only within the current source file, giving them internal linkage.

Standard namespace (`std`)

The C++ Standard Library is inside the `std` namespace.

Without `using namespace std;`:

```
std::cout << "Hello";
std::cin >> x;
```

With:

```
using namespace std;

cout << "Hello";
cin >> x;
```

Many developers prefer using `std::` explicitly in larger projects to avoid name collisions.

Advantages of namespaces

- Prevents name conflicts.
- Organizes large projects.
- Makes libraries easier to manage.
- Allows different libraries to use the same function or class names.

Summary

Feature	Example
Define namespace	`namespace MySpace { ... }`
Access member	`MySpace::function();`
Import entire namespace	`using namespace MySpace;`
Import one member	`using MySpace::x;`
Nested namespace	`A::B::func();`
Anonymous namespace	`namespace { ... }`

Interview questions

What is a namespace in C++?
A namespace is a named scope used to organize code and avoid naming conflicts.

What does the `::` operator do?
It is the scope resolution operator used to access members of a namespace, class, or global scope.

Why is `std::cout` written with `std::`?
Because `cout` belongs to the `std` namespace.

What is the difference between `using namespace std;` and `using std::cout;`?
`using namespace std;` imports all names from `std`.
`using std::cout;` imports only `cout`.

What is an anonymous namespace?
An unnamed namespace whose members are visible only within the current source file.

What is a name conflict in C++?

A name conflict happens when two or more variables, functions, or classes have the same name, and the compiler doesn't know which one you mean.

Example without namespaces (❌ Error)

```
#include <iostream>
using namespace std;

void print() {
    cout << "First print\n";
}

void print() {   // Error
    cout << "Second print\n";
}

int main() {
```

```
    print();
}
```

Error:

redefinition of `print`

The compiler sees two functions with the same name and the same parameters, so it cannot decide which one to use.

Example with namespaces (✅ No Error)

```
#include <iostream>
using namespace std;

namespace Library1 {
    void print() {
        cout << "Library 1\n";
    }
}

namespace Library2 {
    void print() {
        cout << "Library 2\n";
    }
}

int main() {
    Library1::print();
    Library2::print();
}
```

Output:

Library 1
Library 2

Here:

`Library1::print()` means the `print()` function inside `Library1`.
`Library2::print()` means the `print()` function inside `Library2`.

The namespace acts like a family name, so there is no conflict.

Real-life analogy

Imagine your class has two students named Rahul.

If the teacher says:

"Rahul, come here."

Both students may respond.

To avoid confusion, the teacher says:

`ClassA::Rahul`
`ClassB::Rahul`

Now everyone knows exactly which Rahul is being called.

Similarly in C++:

`Math::add()`
`Physics::add()`

Both functions are named `add`, but they belong to different namespaces, so there is no conflict.

In one sentence

A name conflict occurs when two or more identifiers have the same name in the same scope. Namespaces solve this by placing those names into different scopes.

C++ `using namespace`

After learning namespaces, the next topic is `using namespace`.

What is `using namespace`?

Normally, to access members of a namespace, we write:

`NamespaceName::member`

Example:

```
#include <iostream>

int main() {
    std::cout << "Hello";
}
```

Here:
`std` → namespace
`cout` → member
`::` → scope resolution operator

Why do we use `using namespace`?

If we have to write `std::` many times, the code becomes repetitive.

Instead, we can write:

```
using namespace std;
```

Then we don't need `std::` repeatedly.

Example 1

Without `using namespace`:

```
#include <iostream>

int main() {
    std::cout << "Hello\n";
    std::cout << "Welcome";
}
```

Output:

Hello
Welcome

With `using namespace`:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello\n";
    cout << "Welcome";
}
```

Output:

Hello
Welcome

Notice that `std::` is no longer needed.

How does it work?

`using namespace std;` means

"Bring all names from the `std` namespace into the current scope."

Now you can directly use:

- `cout`
- `cin`
- `string`
- `vector`
- `endl`

instead of:

- `std::cout`
- `std::cin`
- `std::string`
- `std::vector`
- `std::endl`

Example 2

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name;

    cout << "Enter your name: ";
    cin >> name;

    cout << "Hello " << name;
}
```

Output:

Enter your name: Sourabha
Hello Sourabha

Using a specific member

Instead of importing everything:

```
using namespace std;
```

you can import only one member.

Example:

```
#include <iostream>
```

```
using std::cout;

int main() {
    cout << "Hello";
}
```

Only `cout` is imported.

Another example:

```
#include <iostream>

using std::cin;
using std::cout;

int main() {
    int x;

    cout << "Enter number: ";
    cin >> x;

    cout << x;
}
```

Why avoid `using namespace std;` in large projects?

Imagine two namespaces.

```
namespace A {
    int value = 10;
}

namespace B {
    int value = 20;
}
```

If you write:

```
#include <iostream>
using namespace std;
using namespace A;
using namespace B;

int main() {
    cout << value;
}
```

The compiler gets confused.

Error: reference to `value` is ambiguous

Both namespaces contain `value`.

Correct way:

```
cout << A::value;
cout << B::value;
```

Best practice

Small programs (learning)
✅ Fine to use `using namespace std;`

Large projects / interviews
Prefer:

- `std::cout`
- `std::cin`
- `std::string`
- `std::vector`

It avoids name conflicts and makes code clearer.

Summary

Statement	Meaning
`std::cout`	Access `cout` from the `std` namespace
`using namespace std;`	Import all members of `std`
`using std::cout;`	Import only `cout`
`::`	Scope resolution operator

Interview questions

1. What does `using namespace std;` do?
   It imports all names from the `std` namespace into the current scope, allowing you to use them without writing `std::`.

2. Is `using namespace std;` mandatory?
   No. You can always use `std::cout`, `std::cin`, etc., instead.

3. Why is `using namespace std;` discouraged in large projects?
   Because it can introduce name conflicts and make code less clear when different namespaces contain members with the same name.

Next topic (recommended)

➡️ Scope Resolution Operator (`::`) — you'll learn how it is used not only with namespaces, but also with classes, global variables, constructors, and more.